

文档四 · Python 内置函数手册

打印设置：A4、边距默认。用法：配合 futurecoder 学习，学到哪查到哪；每天熟记 5 个函数即可。

第 1 页 · 理念篇：先搞懂函数是什么，再记函数

1. 函数 = 一台加工机器

```
len( "hello" )    →    5
|      |          |
|      |——原料（参数）      |——产品（返回值）
|      |          |
|——机器（函数名）
```

函数就是：**原料进去（参数）** → **机器加工** → **产品出来（返回值）**。你只关心"喂什么进去、吐什么出来"，不用管机器内部怎么转。

2. 三条黄金规律

✦ 规律①：**函数名就是它的意思**。print=打印、input=输入、len=长度(length)、sum=求和、sorted=排序、type=类型。**看懂英文就懂函数**，所以文档二的单词要先背。

✦ 规律②：**括号是"启动开关"**。len 只是机器本身，len("hi") 才开工。函数名后面跟 () 才是"调用"。

✦ 规律③：**函数 vs 方法**。函数是 len(x)（函数在括号前）；方法是 x.split()（挂在对象后面，点号调用）。同一个意思：加工数据，只是"手拿工具"和"工具长在对象身上"的区别。

顺手搞懂：变量 = 贴了标签的盒子

```
age = 19
```

意思：造一个盒子，贴上"age"的标签，把数字 19 放进去。以后写 age 就等于是 19。

取名规律：**见名知意**。age=年龄、name=名字、disk_usage=磁盘使用率。英文小写，多个词用下划线连接（蛇形命名）。

3. 最重要的两个函数（第一天就认识）

```
print("你好")          # 把东西显示到屏幕上
print("a", "b", sep="-") # a-b (sep=指定分隔符，默认空格)
print("第一行", end="") # end= 指定结尾，默认换行
```

```
name = input("你叫什么：") # 等用户敲键盘，敲的内容存进 name
# ⚠ input 返回的永远是字符串！"19" 是字符串不是数字，算数前要 int("19")
```

4. 类型转换家族 = "工厂函数"

str / int / float / bool / list / tuple / dict / set 这八个函数，都是**工厂**：把一种类型的东西，加工成另一种类型。

```
int("42")      # → 42 数字（字符串变整数）
str(42)         # → "42" 字符串（整数变字符串，拼接前必须做这步）
float("3.5")    # → 3.5 小数
list("abc")     # → ["a", "b", "c"]（字符串拆成列表）
bool(0)         # → False（0和空东西是假，其他都是真）
dict(name="小刘") # → {"name": "小刘"}
```

函数清单（按使用频率排序）

第 1 组 · 输入输出（第一优先级）

函数	作用	示例	注意
<code>print()</code>	打印到屏幕	<code>print("你好")</code> → 你好 <code>print(1, 2, sep=",")</code> → 1,2	sep=分隔符；end=结尾（默认换行）
<code>input()</code>	读键盘输入	<code>name = input("名字: ")</code>	⚠️ 返回的永远是字符串

第 2 组 · 类型工厂（第二优先级）

函数	作用	示例	注意
<code>str()</code>	转字符串	<code>str(19)</code> → "19"	拼接前必须转
<code>int()</code>	转整数	<code>int("42")</code> → 42	转不了会报 ValueError
<code>float()</code>	转小数	<code>float("3.14")</code> → 3.14	同上
<code>bool()</code>	转真假	<code>bool(0)</code> → False; <code>bool("")</code> → False	0、空字符串、None、空列表 = False
<code>list()</code>	转列表	<code>list("ab")</code> → ["a", "b"]	可把 range 变列表
<code>tuple()</code>	转元组（定死的列表）	<code>tuple([1,2])</code> → (1, 2)	元组不能修改
<code>dict()</code>	转/建字典	<code>dict(a=1)</code> → {"a": 1}	键值对
<code>set()</code>	转集合（去重）	<code>set([1,1,2])</code> → {1, 2}	自动去重、无顺序

第 3 组 · 长度与统计

函数	作用	示例	注意
<code>len()</code>	量长度（length）	<code>len("hello")</code> → 5 <code>len([1,2])</code> → 2	字符串、列表、字典都能量
<code>sum()</code>	求和	<code>sum([1,2,3])</code> → 6	只对数字列表
<code>min()</code> / <code>max()</code>	最小 / 最大	<code>max([3,1,2])</code> → 3	字符串比的是字母顺序
<code>abs()</code>	绝对值	<code>abs(-5)</code> → 5	
<code>round()</code>	舍入（注意！）	<code>round(3.14159, 2)</code> → 3.14	第二个参数=保留几位小数。⚠️ Python3 是"银行家舍入"（四舍六入五成双）： round(2.5)→2、 round(3.5)→4，遇到 .5 时往偶数靠

第 4 组 · 序列工具（循环的好搭档）

函数	作用	示例	注意
<code>range()</code>	生成数列	<code>range(5)</code> → 0,1,2,3,4 <code>range(1,10,2)</code> → 1,3,5,7,9	参数：起、止(不含)、步长。配 for 用；要列表加 list()
<code>enumerate()</code>	边遍历边编号	<code>for i, x in enumerate(["a","b"]):</code> <code>print(i, x)</code> # 0 a 然后 1 b	同时拿到"第几个"和"是什么"
<code>zip()</code>	拉链配对	<code>list(zip([1,2],["a","b"]))</code> → [(1,"a"),(2,"b")]	两个列表——配对
<code>sorted()</code>	排序（不改变原列表）	<code>sorted([3,1,2])</code> → [1,2,3] <code>sorted(xs, reverse=True)</code> 倒序	返回新列表，原列表不变
<code>reversed()</code>	反转	<code>list(reversed([1,2]))</code> → [2,1]	要包一层 list() 才能看
<code>map()</code>	批量加工	<code>list(map(int, ["1","2"]))</code> → [1, 2]	把函数用到每个元素上
<code>filter()</code>	批量筛选	<code>list(filter(lambda x: x > 2, [1,3]))</code> → [3]	留下"条件为真"的元素

第 5 组 · 检查工具（排错用）

函数	作用	示例	注意
<code>type()</code>	看类型	<code>type(5)</code> → int; <code>type("a")</code> → str	搞不清变量是什么类型时就问它
<code>isinstance()</code>	判断是不是某类型	<code>isinstance(5, int)</code> → True	返回 True/False
<code>dir()</code>	列出对象所有属性和方法	<code>dir("abc")</code> # 字符串能干什么全在这	忘了方法名就用它查
<code>help()</code>	看说明书	<code>help(len)</code>	英文的，配合文档二单词看

第 6 组 · 数学与其他

函数	作用	示例	注意
<code>pow()</code>	幂运算	<code>pow(2, 3)</code> → 8（2的3次方）	等于 2**3
<code>divmod()</code>	商和余数一起给	<code>divmod(10, 3)</code> → (3, 1)	(商, 余数)
<code>chr()</code> / <code>ord()</code>	字符 ↔ 编码互转	<code>ord("A")</code> → 65; <code>chr(65)</code> → "A"	知道即可
<code>id()</code>	对象身份证号	<code>id(x)</code>	每个对象唯一，知道即可
<code>format()</code>	格式化输出	<code>format(3.14159, ".2f")</code> → "3.14"	控制小数位数
<code>all()</code> / <code>any()</code>	一批条件：全真 / 有一个真	<code>all([x > 0 for x in xs])</code> 都大于0吗 <code>any([x > 100 for x in xs])</code> 至少有一个吗	批条件判断，返回 True/False
<code>repr()</code>	看值的"原始表示"	<code>repr("hi")</code> → "'hi'"（带引号）	调试时区分类型用，知道即可

文件操作（必会，运维天天用）

```
# 打开文件 (r=读 w=写覆盖 a=追加; encoding 指定编码防中文乱码)
f = open("a.txt", "r", encoding="utf-8")
content = f.read()           # 全读进来（一个字符串）
lines = f.readlines()        # 按行读（一个列表，每行一项）
f.write("新内容")            # 写入
f.close()                    # ⚠️ 用完必须关！

# 推荐写法: with 自动关闭，不用手动 close
with open("a.txt", "r", encoding="utf-8") as f:
    content = f.read()
```

字符串方法（10 个必会，字符串是运维处理的头号对象）

这些都是"方法"——挂在字符串后面用点号调用，作用类似函数。

方法	作用	示例	注意
split()	切开成列表	"a,b,c".split(",") → ["a", "b", "c"]	★最常用，解析日志靠它
join()	列表拼成字符串	",".join(["a", "b"]) → "a,b"	split 的逆操作
strip()	去首尾空白	" hi ".strip() → "hi"	读文件后必做：去掉换行符
replace()	替换	"a-b".replace("-", "_") → "a_b"	全替换
upper() / lower()	大写 / 小写	"abc".upper() → "ABC"	
find()	找子串位置	"hello".find("ll") → 2	找不到返回 -1（不报错）
startswith() / endswith()	判断开头/结尾	"app.log".endswith(".log") → True	★判断文件类型常用
count()	数出现次数	"aaa".count("a") → 3	
format() / f-string	格式化插入变量	f"年龄: {age}岁"	★f-string 最简单，推荐
isdigit()	是不是纯数字	"123".isdigit() → True	int() 转换前先问一句

- ① **输入输出**: print 往外说, input 往里听 (都是字符串!)
- ② **类型工厂**: str int float bool list tuple dict set, 八个工厂八个型
- ③ **量一量**: len 长度、sum 求和、min 小、max 大、abs 绝对值、round 注意银行家舍入
- ④ **循环搭档**: range 数列、enumerate 编号、zip 配对、sorted 排序
- ⑤ **排错工具**: type 看类型、dir 看菜单、help 看说明书
- ⑥ **文件三连**: open 开门 → read/write 干活 → close 关门 (用 with 自动关)
- ⑦ **字符串十招**: split 切、join 拼、strip 洗、replace 换、find 找、count 数、upper/lower 大小写、startswith/endswith 看头尾、f-string 塞变量、isdigit 验数字
- ⚠ **eval() / exec() 是危险函数**: 它们能把字符串当代码执行。网上很多新手教程用它图方便, 但真实项目里禁用——尤其绝不能对用户输入使用, 会被黑客注入恶意代码。见到它绕开走。